



CS100 Recitation 4

Pointers & Arrays

@lichf2025@shanghaitech.edu.cn
@heyx2025@shanghaitech.edu.cn

Contents

Contents

- 1 Pointers ->
- 2 Arrays ->
- 3 C-Style Strings ->
- 4 Dynamic Memory Management ->

1. Pointers

1. Pointers

- 1 Pointer Types ->
- 2 Null Pointers and Null Pointer Values ->
- 3 Undefined Dereferencing ->
- 4 [] Operator ->
- 5 const Qualification ->

1. Pointers

1.1. Pointer Types

- Basics
- Legal pointer type conversion
- Pointer aliasing
 - Strict aliasing
- `void *`

1.1. Pointer Types

1.1.1. Basics

为什么我们需要指针？

```
int main(void) {  
    int n = 42;  
    int* p = &n; // p 是一个指针，指向 n 的地址  
    *p = 100; // 通过指针 p 修改 n 的值  
    return 0;  
}
```

函数参数：形式参数

```
void eat(int hungry_state) {  
    hungry_state = 0; // 修改的是形参 hungry_state 的副本，而不是实参 hungry_state 的值  
}  
int main(void) {  
    int hungry_state = 1;  
    eat(hungry_state); // 没有任何作用  
}
```

函数参数：指针参数

```
void eat(int* hungry_state) {  
    *hungry_state = 0; // 修改实参的值  
}  
int main(void) {  
    int hungry_state = 1;  
    eat(&hungry_state);  
}
```

1.1. Pointer Types

1.1.1. Basics

- 指针的类型是 `PointeeType *`。
- 对于两个不同的类型 `T1` 和 `T2`，指针类型 `T1 *` 和 `T2 *` 是**不同的类型**，尽管它们可能指向同一个内存地址。

```
int i = 42;  
float *fp = &i;  
++*fp; // 未定义行为。这并非对 i 进行自增操作。
```

- 在 C 语言中，不同类型的指针可以（可能会产生警告）相互隐式转换。这是**极不安全的**，并且在 C++ 中属于错误。

1.1. Pointer Types

1.1.2. Legal Pointer Type Cconversion

当试图解引用一个类型为 T_1^* 的指针，而它实际指向的是一个 T_2 类型的对象时，这**几乎总是**未定义行为，**除非**满足以下情况：

1. T_2 与 T_1 是 **兼容类型(compatible type)**（即足够相似的类型）。

```
int x = 1; int *p = &x; printf("%d\n", *p);  
// *p 是 int 类型的左值表达式；与 int 兼容（即相同）
```

2. T_2 是与 T_1 兼容的类型，但带有 `const`、`volatile` 或 `restrict` 限定符的版本。

```
int x = 1; const int *p = &x; printf("%d\n", *p);
```

1.1. Pointer Types

1.1.2. Legal Pointer Type Cconversion

3. `T2` 是与 `T1` 兼容的类型的有符号或无符号版本。

```
int x = 1; unsigned int *p = (unsigned int*)&x; printf("%u\n", *p );
```

4. `T2` 是一个字符类型 (`char`、`signed char` 或 `unsigned char`)。

```
int x = 65; char *p = (char *)&x; printf("%c\n", *p);
```

5. 等等。

更多详细信息，请参阅兼容类型 ->。(超出范围)

1.1. Pointer Types

1.1.3. Pointer Aliasing (指针别名) - Beyond the Scope

在 C 和 C++ 中，当多个指针指向同一内存区域时，会发生**别名 (aliasing)**。

```
void foo(double* a, double* b, int n) {  
    for (int i = 0; i < n; i++) {  
        b[i] += a[i];  
    }  
}
```

- 如果 `a` 和 `b` 指向不同的内存块，代码是完全可并行化的，并且可以进行优化。
- 然而，如果我们以 `foo(a, a + 1, n - 1);` 的方式调用 `foo`，优化可能会出现问题。

1.1. Pointer Types

1.1.3. Strict Aliasing (严格别名) - Beyond the Scope

- C/C++ **编译器假定 (compilers assume)** **不同类型**的指针永远不会指向同一内存位置，即它们彼此不是别名。
 - “不同类型”的具体细节在“**合法指针转换 (Legal Pointer Conversion)**”章节中提及。
 - 严格别名规则 (strict aliasing rule) 在优化级别 2 及更高级别默认启用。
- **简言之 (In short)**，即使我们**从未**将同一个指针传递给函数，如果函数的两个参数**可能 (potentially)**接受指针别名，编译器也不会执行某些优化以确保正确性。

1.1.3. Strict Aliasing (严格别名) - Example

编译器无法将代码优化成如下所示的形式，因为 `ptr1` 和 `ptr2` 可能互为别名。

```
int foo(int* ptr1, int* ptr2){  
    *ptr1 = 10;  
    *ptr2 = 11;  
    return *ptr1;  
}
```

```
int foo(int* ptr1, int* ptr2){  
    *ptr1 = 10; *ptr2 = 11;  
    return 10;  
}
```

在这种情况下，该优化是正确的。

```
int foo(int* ptr1, double* ptr2){  
    *ptr1 = 10;  
    *ptr2 = 21.02;  
    return *ptr1;  
}
```

```
int foo(int* ptr1, double* ptr2){  
    *ptr1 = 10; *ptr2 = 21.02;  
    return 10;  
}
```

1.1. Pointer Types

1.1.4. Pointers type: `void *`

指向**任何类型对象**的指针都可以隐式转换为 `void *`，反之亦然（vice versa）。

- 在 C++ 中，从 `void *` 到任何其他指针类型的转换必须显式进行。

混乱之处在于：

```
int n = 1  
int* p = &n;  
void* pv = p; // int* to void*  
int* p2 = pv; // void* to int*
```

1.1. Pointer Types

1.1.4. Pointers type: `void *`

`void *` 类型在 C 类型系统中引入了一个弱点，但在传递**未知类型对象**给函数时可能非常有用。以下是一些常见用例：

- `malloc` -> 返回 `void *`。返回的指针应适用于任何对象类型。
 - 此外，不应强制转换 `malloc` 的结果。参见 [StackOverflow](#) ->。
 - 使用 `int* p = malloc(size);` 而非 `int* p = (int *)malloc(size);`
- `qsort` -> 要求用户提供的回调函数接受两个 `const void*` 参数。
 - `void qsort( void* ptr, size_t count, size_t size, int (*comp)(const void*, const void*));`
- `*pthread_create` -> 要求用户提供的回调函数接受并返回 `void *`。

不需要对malloc返回值强制转换

In [this question](#), someone suggested in a [comment](#) that I should **not** cast the result of `malloc`.
i.e., I should do this:

```
int *sieve = malloc(sizeof(*sieve) * length);
```

rather than:

```
int *sieve = (int *) malloc(sizeof(*sieve) * length);
```

Why would this be the case?

1. Pointers

1.2. Null Pointers and Null Pointer Values

- Null pointers
- Null pointer values
- `NULL` and `nullptr`

1.2. Null Pointers and Null Pointer Values

1.2.1. Null Pointers

- 空指针（Null Pointers）是指不指向任何对象的指针。
- 空指针与任何指向**对象**的指针比较时结果都不相等。
- 初始化或赋值：可通过空指针常量（值）实现。

1.2. Null Pointers and Null Pointer Values

1.2.1. Null Pointers

Usage:

- 表示一个对象的缺失。
- 表示其他类型的**错误条件**；例如，如果分配失败，`malloc` 会返回一个空指针。

```
int* ptr = malloc(sizeof(int));  
if (ptr == NULL){ // 或者 if(!ptr)  
    printf("Allocation failed\n");  
}  
free(ptr);
```

通常，接收指针参数的函数几乎总是需要**检查该值是否为空**，并以不同的方式处理这种情况。

1.2. Null Pointers and Null Pointer Values

1.2.2. Null Pointers Value

- 空指针值（Null pointer value）是指针类型的“零”值。
- 每种指针类型都有一个特殊值，称为**该类型的空指针值**。
- 两个空指针值比较时相等，这是由比较运算符 `->` 定义的。

1.2. Null Pointers and Null Pointer Values

1.2.2. Null Pointers Value

空指针常量的分类：

- `NULL` ->: 该宏定义可从多个标准库头文件中获取（如 `<stddef.h>`, `<stdio.h>`, `<stdlib.h>`）

```
int* ptr = NULL;
```

- 值为零的整型常量

```
double* ptr_1 = 0;  
double* ptr_2 = 2 - 2;
```

1.2. Null Pointers and Null Pointer Values

1.2.3. NULL and nullptr

- 宏 `NULL` 是一个 **实现定义(implementation-defined)** 的空指针常量，其可能为：
 - 值为 `0` 的整型常量表达式（例如 `2 - 2`）
 - 值为 `0` 且强制转换为 `void *` 类型的整型常量表达式（例如 `(void *)0` 或 `0x00000000`）
 - [C23 起] 预定义常量 `nullptr` ->
- `nullptr`: **类型安全**
 - 属于 `nullptr_t` 类型：一种独特的类型，本身不是指针类型，仅有一个有效值 `nullptr`
 - 可转换为指针类型或 **bool** 类型：分别转换为该类型的空指针值或 `false`

1.3.Undefined Dereferencing

尝试解引用一个**不指向实际对象**的指针会导致**未定义行为**，这通常被视为**严重的运行时错误**。可能发生这种情况的场景包括：

- **空指针**
- **野指针（未初始化的指针）**： `int* p;`
- **悬垂指针**：指向刚刚被释放的内存的指针。
 - 曾经指向通过 `malloc` 分配但随后已被 `free` 的内存。
 - 指向已超出作用域的变量的指针，例如：

```
int* foo(int n) { return &n; }
```

- **指向另一个无效或无意义的地址**： `int* p = (int*)123;`

UB行为的危害

未定义行为（Undefined Behavior, UB）可能导致程序出现各种不可预测的后果，包括但不限于：

- 程序崩溃（Crash）（Runtime error）
- 数据损坏
- 安全漏洞
- 不可预见的输出
- 程序挂起或进入无限循环
- 看似正常的行为，但在不同的编译器或编译选项下表现不同

UB行为的危害

```
#include <stdio.h>

int main(void) {
    int x[4] = {0, 1, 2, 3};
    int y[4] = {0, 1, 2, 3};
    x[4] = 10; // Undefined behavior: 改了非法地址x[4]
    printf("address of x:%x\n", (unsigned int)&x);
    printf("address of x[4]:%x\n", (unsigned int)&(x[4]));
    printf("address of y:%x\n", (unsigned int)&y);
    printf("%d\n", y[0]); // 你虽然没有改y的值, 但是y[0]可能被改了
    return 0;
}
```

```
address of x:1f692070
address of x[4]:1f692080
address of y:1f692080
y[0] = 10
```

如何检测UB行为

- (Linux)使用编译器选项启用运行时检查 (如 `-fsanitize=undefined -fsanitize=address`)。

```
gcc ub.c -o ub -fsanitize=undefined -fsanitize=address -g
```

```
lichf1@vsp-lab1 ~/CS100> ./ub
ub.c:6:4: runtime error: index 4 out of bounds for type 'int [4]'
ub.c:6:8: runtime error: store to address 0x705422800030 with insufficient space for an object of type 'int'
0x705422800030: note: pointer points here
 03 00 00 00  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00 00 00 00 00 00 00 01 00 00 00  02 00 00 00
  ^
=====
==160534==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x705422800030 at pc 0x5d76dff2356c bp 0x7ffdf0a968b0 sp 0x7ffdf0a968b0
WRITE of size 4 at 0x705422800030 thread T0
#0 0x5d76dff2356b in main /home/lichf1/CS100/ub.c:6
#1 0x705424c2a1c9 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
#2 0x705424c2a28a in __libc_start_main_impl ../csu/libc-start.c:360
#3 0x5d76dff231c4 in _start (/home/lichf1/CS100/ub+0x11c4) (BuildId: a79c4ceb79ac74123d0ecc793ef20943935186ae)

Address 0x705422800030 is located in stack of thread T0 at offset 48 in frame
#0 0x5d76dff23298 in main /home/lichf1/CS100/ub.c:3
This frame has 2 object(s):
  [32, 48) 'x' (line 4) <== Memory access at offset 48 overflows this variable
  [64, 80) 'y' (line 5)
```

访问非法地址的位置: ub.c 的第六行

出问题的地址源自哪个语句: ub.c的第三行

1.4. Subscript Operator []

语法:

- 形式1: 指针表达式 [整数表达式]
- 形式2: 整数表达式 [指针表达式]

定义: 表达式 `E1[E2]` 完全等价于 `*((E1) + (E2))`。

- 若任一形式中的指针表达式为数组表达式, 则会进行**隐式数组到指针转换**, 生成指向数组首元素的指针。
- `arr[N]` 等价于数组 `arr` 的第 `N` 个元素, 这是指针算术运算 (特别是指针与整数的加法) 的直接结果, 符合上述定义。
- 对于定义为 `int a[3] = {1, 2, 3};` 的数组, 表达式 `a[2] == 2[a]` 的计算结果为真。

1.5. `const` Qualification

- `const` Semantics
- Low-level `const`ness of Pointers & Top-level `const`ness of Pointers

1.5. const Qualification

1.5.1. const Semantics(语义)

`const` 限定符的语义仅适用于具有标识的左值表达式。此类表达式不可修改，尤其不可**赋值**。

```
const int n = 1; // const 限定类型的对象
n = 2;          // 错误: n 是 const 限定类型

int x = 2;      // 非限定类型的对象
const int* p = &x;
*p = 3;         // 错误: 左值 *p 是 const 限定类型

int const ci;   // 未初始化
ci = 5;         // 错误: ci 是 const 限定类型
```

使用 `const` 限定类型声明的对象可能会被编译器放置在**只读内存**中。此外，如果一个 `const` 对象的地址在程序中从未被获取，该对象可能根本不会被存储。

1.5. const Qualification

1.5.2. Low-level and Top-level const-ness of Pointers

- **指向 const 的指针**：底层 const（const 限定适用于被指向的对象）
 - 指向 const 的指针可以指向非 const 对象。
 - 指向非 const 类型的指针可以隐式转换为指向同一类型的 const 限定版本的指针。然而，反向转换需要显式类型转换。

```
int n = 42; int* p = &n;  
const int* cp = p; // 正确：添加限定符 (int 到 const int)  
p = cp;           // 错误：丢弃限定符 (const int 到 int)  
p = (int *)cp;    // 正确：使用类型转换
```

“ 但是，使用 `p` 来修改 `n`（它所指向的对象）的值会导致未定义行为。

1.5. `const` Qualification

1.5.2. Low-level and Top-level `const`-ness of Pointers

- `const` 指针：顶层 `const` (`const` 限定适用于指针本身)
 - 在这种情况下，被指向的对象是可修改的，但指针本身不可修改。

C语言中多级指针的 `const` 修饰

从右往左读的原则

“ **阅读规则**：从变量名开始，向右看，再向左看。

```
int** p const;
```

阅读顺序：

p 是一个 “const pointer”

*p 是一个指针

**p 是一个 “const pointer”，指向 int*

从右往左读的原则

“ **阅读规则**：从变量名开始，向右看，再向左看。

```
int* const *p;
```

阅读顺序：

p 是一个指针

p 是一个 “const pointer”

指向 int

⇒ “pointer to const pointer to int”

Exercise 1

What level of `const`-ness is specified by the following `const`-qualified pointer types?

- `int** const p`
- `int* const* p`
- `int const** p`
- `int* const* const p`

Exercise 1

- **int** const**: A **const** pointer to a pointer to an **int**.
 - The pointer itself is constant and cannot change, but the **int** it points to can be modified.
- **int* const***: A pointer to a **const** pointer to an **int**.

```
int a = 42;  
int* const p = &a; // const pointer to int  
int* const* ptr = &p; // pointer to a const pointer
```

- The **const** pointer cannot change, but the **int** it points to can.
- **int const**** or **const int****: A pointer to a pointer to a **const int**.
 - The **int** is **const** and cannot be modified, but the pointers can change.
- **int* const* const**: A **const** pointer to a **const** pointer to an **int**.
 - Both pointers are **const** and cannot change, but the **int** can be modified.

2. Arrays

2. Arrays

- 1 Variable-Length Array (VLA) ->
- 2 Array to Pointer Conversion ->

2. Arrays

2.1. Variable-Length Array (VLA)

区别于常量大小的数组：

```
int n[10];  
char str[] = "abc";  
Type arr[N]; // `N` 是一个在编译时值已知的常量表达式。  
...
```

如果数组的大小在**运行时**确定，则其为变长数组 (VLA)：

```
int n;  
scanf("%d", &n);  
int a[n];
```

- 变长数组的生命周期在其声明的作用域结束时终止。
- 每次控制流到达该声明时，都会重新计算大小表达式并重新分配数组。
- 在后续执行同一代码时，变长数组可能会**以不同大小**重新分配。

2. Arrays

2.1. Variable-Length Array (VLA)

```
int n;  
scanf("%d", &n);  
int a[n];
```

- 变长数组（VLA）是在 **C99** 标准中被引入 C 语言的，但自 **C11** 起，编译器不再被强制要求支持此特性。
- **我们不推荐也不允许使用变长数组。**
 - 你应当改用 `malloc` 或 `calloc` 进行动态内存分配。
- 在 C++14 的讨论过程中，曾考虑过引入变长数组，但最终**投票否决**了该提案。
 - 更多细节可参考 Gkxx 的 CSDN 博客 ->。

2. Arrays

2.2. Array to Pointer Conversion

- `sizeof` Operator on Arrays
- Decay From Arrays to Pointers

2.2. Array to Pointer Conversion

2.2.1. `sizeof` Operator on Arrays

必须强调的是，数组和指针是两种不同的类型，**尽管存在从数组到指针的常见转换。**

“ 回顾： `sizeof` 是一个返回类型字节大小的运算符。 `sizeof(type)` 的返回类型是 `size_t`，这是一种由实现定义的无符号整数类型。

注意： 对于变长数组（VLA）， `sizeof` 是在运行时计算的。

2.2. Array to Pointer Conversion

2.2.1. sizeof Operator on Arrays: Example

```
#include <stdio.h>
int main(void){
    int a[] = {1, 2, 3, 4, 5};
    printf("Size of the array: %zu\n", sizeof(a));
    // 20 (5 elements * 4 bytes each)
    printf("Size of a pointer to the array: %zu\n", sizeof(&a)); // 8
    printf("Size of a pointer to the first element: %zu\n", sizeof(&a[0])); // 8
}
```

2.2. Array to Pointer Conversion

2.2.2. Decay From Arrays to Pointers

“退化”（Decay）指的是数组隐式转换为指向其首元素的指针这一过程。

- 类型为 `Type [N]` 的数组会退化为类型为 `Type *` 的指针。
- 表达式 `p = &arr[0]` 可以简写为 `p = arr`。
- 表达式 `*arr` 等价于 `*(&arr[0])` 或 `arr[0]`。

这一转换由 C 语言标准定义。

2.2. Array to Pointer Conversion

2.2.2. Decay From Arrays to Pointers

任何数组类型的左值表达式（例如，数组的标识符/名称）在用于以下情况**之外**的任何上下文时，都会发生此转换：

- 作为取地址运算符 `&` 的操作数：
 - `(&arr = arr)`但是`(arr + 1 != &arr + 1)`
- 作为 `sizeof` 的操作数
 - `sizeof(ptr) = 8` (在64位系统上), 但是`sizeof(arr) = 20` (假设arr有5个int元素)
- 作为用于数组初始化的字符串字面量
 - `char s[] = "hi";` 会复制字符串, 而不是指针
- 作为 `typeof` 和 `typeof_unqual` 的操作数 (自 C23 起), `*[超出范围]`

在上述这些上下文中，数组**不会**退化为指针。(数组和指针是不同的类型)

3. C-Style Strings

3. C-Style Strings

- 1 String I/O ->
- 2 C Strings Library ->

3. C-Style Strings

3.1. String I/O

- Error-Prone Input: `scanf`, `gets`
- Memory-Safe Input: `fgets`
- Output: `puts`

3.1. String I/O

3.1.1. Error-Prone Input: `scanf`, `gets`

- `scanf` -> / `printf` ->: 使用 `"%s"`
 - 使用格式说明符 `"%s"` 的 `scanf` 函数存在内存安全问题，因为它仅接受 `char *` 类型的指针，而未指定数组的长度。
- `gets(char* str)`: 读取字符串时不进行**边界检查**。
 - 该函数自 **C11** 起已从标准中移除。
 - 具有边界检查功能的替代函数是 `gets_s`，但并非所有编译器（包括 GCC）都支持它。

“ `char* gets_s(char* str, rsize_t n);`

`gets_s` 函数最多写入 `n - 1` 个字符，以确保有空位容纳空终止符。

2024Fall quiz1 Q4: Find bugs!

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char buffer[6];    // (1)
    int pass = 0;      // (2)
    printf("Enter the password:\n");
    if (gets(buffer) != NULL) {    // (3)
        if (strcmp(buffer, "CS100") != 0) {    // (4)
            printf("!!! Wrong Password. \n");
        }
        else {
            printf("!!! Correct Password. \n");
            pass = 1;
        }
        if (pass)    // (5)
            printf("!!! You are the root. \n");
    }
    return 0;
}
```

3.1. String I/O

3.1.2. Memory-Safe Input: `fgets` ->

- **Signature:** (Defined in header `<stdio.h>`)

```
char* fgets(char* str, int count, FILE* stream);
```

- `fgets` 比其他输入函数更具可移植性、通用性和安全性，因为它包含边界检查：
 - 它从指定流中最多读取 `count - 1` 个字符。
 - 当遇到换行符 (`\n`) 或文件结束符 (`EOF`) 时停止读取。
 - 成功时返回指向 `str` 的指针；失败时返回空指针。

Example:

```
char str[100];  
fgets(str, sizeof(str), stdin);
```

3.1. String I/O

3.1.3. Output: puts

Signature: (Defined in header `<stdio.h>`)

```
int puts(const char* str);
```

- `puts` 函数将空终止字符串 `str` 中的每个字符写入输出流 `stdout`，并在末尾额外添加一个换行符 (`\n`)。
- `str` 的终止空字符 (`\0`) 不会被写入输出。

Question:

为什么 `puts` 的参数声明为 `const`，而 `fgets` 的参数却不是？

3.2. C Strings Library

- Review of In-class Implementation of `strlen`
- Implementation of `strcpy`**
- Overview of C Strings Library

3.2. C Strings Library

3.2.1. Review of In-Class Implementation of `strlen`

```
size_t my_strlen(const char* str) {  
    size_t ans = 0;  
    while (*str++ != '\0')    // How about while (++str != '\0')?  
        ++ans;  
    return ans;  
}
```

- **运算符优先级**: 后缀递增运算符 `++` 的优先级高于解引用运算符 `*`。

3.2. C Strings Library

3.2.1. Review of In-Class Implementation of `strlen`

函数 `strlen(s)` 每次被调用时都会扫描整个字符串 `s`。

```
for (size_t i = 0; i < strlen(s); ++i)
    // ...
```

在此循环中，条件 `i < strlen(s)` 在每次迭代结束时都会被重新求值。由于 `strlen(s)` 每次被调用时都需要完整遍历整个字符串，这会导致**性能非常低下**，时间复杂度为 $O(n^2)$ 。

3.2. C Strings Library

3.2.1. Review of In-Class Implementation of `strlen`

为提升性能，应将其修改为：

```
for (size_t i = 0, n = strlen(s); i < n; ++i)
    // ...
```

通过预先计算字符串长度并将其存储在变量 `n` 中，循环可以在线性时间 $O(n)$ 内完成执行。

3.2. C Strings Library

3.2.1. Review of In-Class Implementation of `strlen`

```
for (int i = 0; i < strlen(s); ++i)
    // ...
```

为什么编译器会对 `i < strlen(s)` 发出警告？

- `strlen` 的返回类型是 `size_t`，这是一种无符号整数类型。
- 当在 `int` 和 `size_t` 之间执行操作或比较时，`int` 值会被隐式转换为 `size_t`。
 - 因此，如果 `i` 为负数（例如 `-1`），比较 `-1 < strlen(s)` 几乎肯定会被判定为 **false**（假）。

*** 应避免混合使用有符号和无符号整数。**

3.2. C Strings Library

3.2.2. Implementation of `strcpy` ->

- **Signature:** (Defined in header `<string.h>`)

```
char *strcpy(char* dest, const char* src);
```

- **Behavior:**

- `strcpy` 函数将以 `src` 指向的空终止字节字符串（包括空终止符 `\0`）复制到以 `dest` 指向其首元素的字符数组中。
- 在以下情况下，行为是未定义的：
 - `dest` 数组不够大，无法容纳 `src` 的内容。
 - 字符串存在重叠。
 - `dest` 不是指向字符数组的指针，或 `src` 不是指向空终止字节字符串的指针。

Exercise 2: Implement Your Own `strcpy` Function

```
char *strcpy(char* dest, const char* src);
```

Exercise 2

Most people cannot resist the urge to use **indexing** and thus call `strlen` first:

```
char *strcpy(char *dest, const char *src) {  
    size_t length = strlen(src);  
    for (size_t i = 0; i <= length; ++i)  
        dest[i] = src[i];  
    return dest;  
}
```

However, `strlen` also scans the `src` string.

- Why scan it twice?

Exercise 2

It is possible to scan only once: the termination condition should be **when encountering '\0'**, rather than `i == strlen(src)`.

```
char *strcpy(char *dest, const char *src) {  
    size_t i = 0;  
    while (src[i] != '\0') {  
        dest[i] = src[i];  
        ++i;  
    }  
    dest[i] = '\0';  
    return dest;  
}
```

Exercise 2: A More Direct Approach

You can directly **manipulate pointers** without needing indexing.

- Both `dest` and `src` are local variables within your function, so you can modify them freely!

```
char *strcpy(char *dest, const char *src) {  
    char *ret = dest;  
    while (*src != '\0')  
        *dest++ = *src++;  
    *dest = '\0';  
    return ret;  
}
```

```
char a[100], b[] = "hello";  
strcpy(a, b); // This works fine.
```

3.2. C Strings Library

3.2.3. Overview of C Strings Library

- Null-terminated byte strings (CppReference) ->

4. Dynamic Memory Management

4. Dynamic Memory Management

- 1 malloc, calloc ->
- 2 realloc ->
- 3 free ->
- 4 In-Class Examples ->

4. Dynamic Memory Management

4.1. `malloc` ->, `calloc` ->

```
void* malloc(size_t size);  
void* calloc(size_t num, size_t each_size);
```

- `malloc`: 分配 `size` 字节的**未初始化内存**，这意味着所分配元素的值是不确定的。
- `calloc`: 为 `num` 个对象（每个大小为 `each_size`）分配内存，并**将所有字节初始化为零**。
 - 由于**对齐要求**，实际分配的字节数可能大于 `num * size`。
 - 注意：将所有字节初始化为零**并不能保证**浮点值被设置为 `0.0` 或指针被设置为**空指针值**。

4.1.1. malloc

Try the following code snippet

根据 C 标准，`malloc` 分配的内存包含**不确定的值**，但在实践中，这些值通常表现为零。这仅仅是巧合吗？

一般来说不是！

- 分配的内存可能是**回收**自先前释放的内存，或是操作系统直接提供的。
- 出于安全原因，操作系统通常会返回**清零后的内存**，以避免泄露其他进程的数据。

更多详细信息，您可以查看 StackOverflow 上的这个回答 ->。

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 5;
    int *arr = malloc(n * sizeof(int));
    printf("Array values:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
    free(arr);
    return 0;
}
```

4.2. `realloc` ->: reallocates a specified area of memory

```
void* realloc(void* ptr, size_t new_size);
```

参数 `ptr` 可以是 `NULL`，也可以是由 `malloc`、`calloc` 或 `realloc` 先前分配且**尚未通过调用 `free` 或 `realloc` 释放**的指针。

- 如果 `ptr` 为 `NULL`，则行为等同于 `malloc(new_size)`。
- 如果 `ptr` 不为 `NULL`，则通过以下方式执行分配：
 - 扩展或收缩 `ptr` 指向的现有内存区域，或
 - 分配一个大小为 `new_size` 字节的新内存块，复制新旧大小中较小者所对应的内存内容，并释放旧内存块。

如果可用内存不足，旧内存块不会被释放，并返回空指针。

4.2. malloc, calloc, and realloc

- 为避免内存泄漏，这些函数返回的任何指针都必须使用 `free`（或 `realloc`）进行释放。需要注意的是，原始指针 `ptr` 在重新分配后即失效，后续对其的任何访问都会导致未定义行为——即使重新分配是原地进行的。
- `malloc(0)`、`calloc(0, N)`、`calloc(N, 0)` 和 `realloc(ptr, 0)` 的行为是**由实现定义的**。
 - 这些函数可能不分配内存并返回空指针。
 - 或者，它们可能分配一定量的内存并返回指向其起始地址的指针。
 - 但解引用此类指针会导致未定义行为。
 - 这部分分配的内存**仍必须被释放**，以避免内存泄漏。

4. Dynamic Memory Management

4.3. `free` -> `void free(void* ptr)`

正确操作：

- 释放空指针：无需检查 `ptr != NULL`。

未定义行为：

- 在指针已被 `free` 释放后，继续使用或再次释放该悬垂指针。
- 释放的地址不是由 `malloc`、`calloc`、`realloc` 或 C11 的 `aligned_alloc` -> 返回的指针。
 - 指针必须指向已分配内存块的**起始位置**，而非中间地址。

提醒：

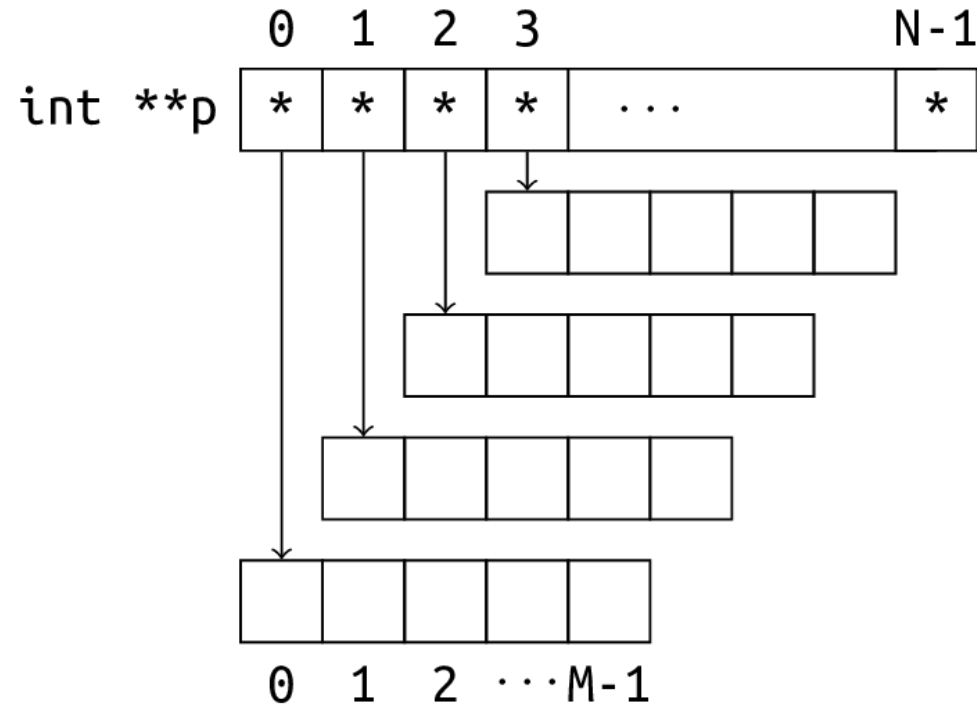
忘记调用 `free` 会导致内存泄漏，这可能使您无法通过在线评测平台（OJ）的测试。

4. Dynamic Memory Management

4.4. In-Class Examples

- Creating a Dynamic 2-Dimensional Array
- Reading a String of Unknown Length
 - `memcpy`
 - `ungetc`

Example 1: Creating a Dynamic 2-Dimensional Array



Example 1: Creating a Dynamic 2-Dimensional Array

Method 1: Create an real 2D Array

```
int **p = malloc(sizeof(int *) * n);
for (int i = 0; i != n; ++i)
    p[i] = malloc(sizeof(int) * m);
for (int i = 0; i != n; ++i)
    for (int j = 0; j != m; ++j)
        p[i][j] = /* ... */
for (int i = 0; i != n; ++i)
    free(p[i]);
free(p);
```

Method 2: Create an 1D array

```
int *p = malloc(sizeof(int) * n * m);
for (int i = 0; i != n; ++i)
    for (int j = 0; j != m; ++j)
        p[i * m + j] = /* ... */
// ...
free(p);
```

Example 2: Reading a String of Unknown Length

假设我们需要读取一个未知长度的非空白字符序列。在这种情况下，我们可以使用 `malloc` 动态分配内存，并根据需要使用 `realloc` 调整其大小。具体步骤包括：

- 动态分配内存以存储字符串
- 如果缓冲区已满，则重新分配一个更大的缓冲区，并将现有数据复制到新缓冲区中
- 完成后释放已分配的内存，避免内存泄漏

Example 2: Reading a String of Unknown Length

// Snippet 1

```
char *read_string(void) {  
    // ignore leading whitespaces  
    char c = getchar();  
    while (isspace(c))  
        c = getchar();  
  
    char *buffer = malloc(INITIAL_SIZE);  
    int capacity = INITIAL_SIZE;  
    int cur_pos = 0;
```

// Snippet 2

```
while (!isspace(c)) {  
    if (cur_pos == capacity - 1) {  
        char *new_buffer = malloc(capacity * 2);  
  
        // copy everything what we have stored  
        // to the new buffer  
        memcpy(new_buffer, buffer, cur_pos);  
  
        // Don't forget to free.  
        free(buffer);  
  
        capacity *= 2;  
        buffer = new_buffer;  
    }  
  
    buffer[cur_pos++] = c;  
    c = getchar();  
}
```

// Snippet 3

```
// Now, c is a whitespace. This is not part of the contents we need.  
ungetc(c, stdin); // Put that whitespace back to the input.  
buffer[cur_pos] = '\\0'; // Remember this!!  
return buffer;  
}
```

Example 2: Reading a String of Unknown Length

memcpy ->

Defined in header `<string.h>`:

```
void* memcpy(void* dest, const void* src, size_t count);
```

- **功能**: 将 `count` 个**字符**从 `src` 指向的内存位置复制到 `dest` 指向的内存位置。返回 `dest`。
- **未定义行为**:
 - 如果 `dest` 没有足够的空间容纳 `count` 个字符
 - 如果 `src`, 和 `dest` 的内存区域**重叠**
 - 如果 `src` 或 `dest` 是无效指针或空指针
- **memcpy 比 strcpy 更快**, 因为 `strcpy` 遇到 `\0` 会停止复制, 而 `memcpy` 会**不检查数据内容**地精确复制指定数量的字节

Example 2: Reading a String of Unknown Length

ungetc ->

```
int ungetc(int ch, FILE* stream); // defined in header <stdio.h>
```

- **ungetc** 将字符 **ch** 推回输入流 **stream** 的缓冲区中。下一次从该流读取操作将会获取到这个被推回的字符。
 - **注意**：此操作不会修改与流关联的外部设备。
- **缓冲区限制**：
 - 仅保证一个字符的推回（缓冲区大小为 1）。如果在读取或重新定位操作之前多次调用 **ungetc**，其行为是**由实现定义的**（implementation-defined），并且可能会失败。
 - 如果进行了多次成功的 **ungetc** 调用，被推回的字符将以**逆序**被读取。

Good luck and enjoy programming!